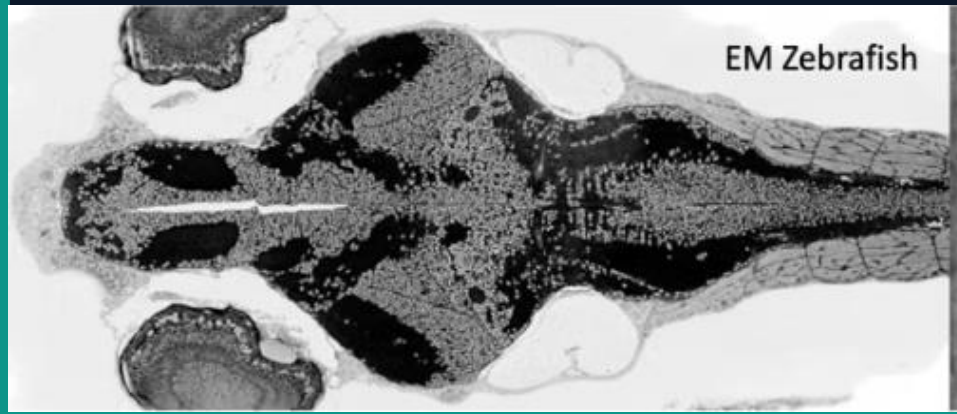
# 3D Nucleus Segmentation

## in Zebrafish Brain Microscopy

Deep Learning Approach Using 3D U-Net · MONAI · PyTorch

**NucMM-Z Dataset · Zebrafish Subset**

Presented by: **Joshua Dakim / University of Eastern Finland / iPSRS**




# Background & Motivation



## Why Zebrafish?

- Vertebrate model organism with tractable brain size
- Entire brain imageable at single-cell resolution
- Homologous neural architecture to mammals
- Transparent embryo — ideal for live imaging
- 100,000+ neurons — each with a nucleus

## Why Nucleus Segmentation?

- Foundation for building complete brain connectomes
- Enables cell counting & morphology quantification
- Critical for tracking neurodegeneration & disease
- Manual annotation of 100K+ nuclei takes years
- Automation unlocks population-scale studies

# Problem Statement



## Volumetric Complexity

- 3D microscopy volumes — not standard 2D images
- GPU memory constraints limit full-volume processing
- Depth-axis anisotropy from imaging physics (PSF blur)
- Patch-based training required for tractability

## Segmentation Difficulty

- Severe class imbalance: ~4.74% foreground voxels
- Touching / overlapping nuclei with indistinct boundaries
- Variable nucleus size, shape, and staining intensity
- Instance labels → binary conversion loses cell identity

## Data & Generalisation

- Only 27 training volumes — extremely small dataset
- Domain shift risk across imaging modalities / labs
- Manual annotation is costly and expert-dependent
- No standardised evaluation across methods

# Dataset Overview



**NucMM Dataset — Nuclear Morphology and Motility Benchmark (Wei et al., 2021)**

| 27 | 27 | $64^3$ | 4.74% |
|---|---|---|---|
| Training Volumes | Validation Volumes | Voxel Patch Size | Foreground Fraction |

## Volume Characteristics

- Format: HDF5 (.h5) with internal key 'main'
- Image dtype: uint8 (0–255 intensity range)
- Label dtype: uint32 — instance IDs per nucleus
- Binarised for segmentation: nucleus vs background
- Patches named by spatial origin: (z, y, x) coords

## Imaging Context

- Electron / fluorescence microscopy of zebrafish brain
- 3D voxel arrays — analogous to CT/MRI volumetric data
- Intensity encodes local light absorption / fluorescence
- PSF blur present — especially along axial (Z) direction
- Source: nucmm-dataset.github.io

# Project Objectives



**01**

## End-to-End 3D Segmentation Pipeline

Design and implement a complete pipeline from raw HDF5 volume loading to binary segmentation output — reproducible, modular, and runnable on free-tier Colab GPU.

**02**

## Patch-Based Training Strategy

Overcome GPU memory limits by training on $64^3$ voxel crops with rich data augmentation (flips, rotations, intensity jitter) to maximise effective dataset size.

**03**

## 3D U-Net with Dice Optimisation

Deploy a MONAI 3D U-Net with residual units, batch normalisation, and Dice loss — specifically suited to the severe class imbalance of nucleus vs background.

**04**

## Quantitative & Qualitative Evaluation

Achieve a validation Dice score ≥ 0.70 and produce interpretable visual comparisons of predictions vs ground truth across multiple Z-slices.

# Methodology



## Data Pipeline

- HDF5 loading (key: 'main')
- Instance labels → binary mask
- Channel dim: (D,H,W) → (1,D,H,W)
- Zero-mean unit-std normalisation
- Random $64^3$ patch crop
- Augmentation: flip (3 axes), 90° rotate, ±10% intensity scale & shift
- batch_size=2, num_workers=2

## Architecture

- 3D U-Net (MONAI implementation)
- spatial_dims = 3
- in/out channels = 1 (grayscale → binary)
- 5 encoder levels: 16→32→64→128→256
- Stride-2 downsampling between levels
- Skip connections — preserves spatial detail
- 2 residual units per level + Batch Norm
- 4,807,968 trainable parameters

## Framework

- PyTorch 2.x — core DL framework
- MONAI 1.5.2 — medical imaging library
- h5py — HDF5 I/O
- NumPy — array processing
- Matplotlib — visualisation
- Google Colab — T4 GPU (15 GB)
- set_determinism(seed=42) for reproducibility

# Training Setup



Loss Function

## Dice Loss

- sigmoid=True — converts logits to probabilities
- Optimises overlap: 2|P∩G| / (|P|+|G|)
- Robust to class imbalance (4.74% fg)
- smooth_nr / smooth_dr = 1e-5

Optimiser

## Adam

- Learning rate: 1e-3 (initial)
- Weight decay: 1e-5 (L2 regularisation)
- Adaptive per-parameter LR updates
- ReduceLROnPlateau: ×0.5 after 5 epochs

Training Strategy

## Patch-Based

- 50 epochs total
- Validate every 2 epochs
- Visualise predictions every 10 epochs
- Best model saved by Dice score

Hardware: NVIDIA T4 GPU · 15 GB VRAM · Google Colab Free Tier | ~20 min training time | Metric: Dice Score (exclude background)

# Results



Best Dice Score

**0.8718**

Validation Set

**Target: ≥ 0.70 ✓ Exceeded by +0.17**


**3D U-Net**

Architecture

**50**

Epochs

**$64^3$**

Patch Size

**4.8M**

Parameters

**T4 15GB**

GPU

# Discussion



## Limitations & Challenges

- Only 27 training volumes — high overfitting risk
- Binary segmentation only — cannot distinguish touching nuclei
- Patch-based: no global spatial context
- Validated on same-domain data only
- Dice score masks errors on small nuclei
- No post-processing (watershed, morphological cleaning)
- Free Colab GPU — compute and session limits
- 50 epochs may not be sufficient for full convergence

## Possible Improvements

- Instance segmentation — separate touching nuclei (Mask R-CNN, StarDist3D)
- Semi-supervised learning — leverage unannotated volumes
- Self-supervised pre-training — reduce labelled data requirement
- Sliding window inference — full-volume prediction at test time
- Watershed post-processing — split merged nucleus predictions
- External dataset validation — cross-lab & cross-species testing
- nnU-Net — automated hyperparameter optimisation
- Longer training (100–200 epochs) with cosine annealing

# Conclusion & Future Work



## Key Takeaways

- End-to-end 3D U-Net pipeline successfully implemented
- Dice score of 0.8718 — substantially above 0.70 target
- Patch-based training feasible on free-tier GPU
- MONAI + PyTorch proved effective for 3D medical imaging
- Augmentation critical to compensating for small dataset

## Future Directions

- Extend to instance segmentation (StarDist3D)
- Apply sliding window inference to full brain volumes
- Validate on mouse cortex subset (cross-species)
- Explore semi-supervised learning for data efficiency
- Integrate with connectomics downstream analysis

**Acknowledgements & References**

**Dataset:** Wei et al. (2021). NucMM Dataset. MICCAI. nucmm-dataset.github.io **Architecture:** Ronneberger et al. (2015). U-Net, MICCAI. Çiçek et al. (2016). 3D U-Net, MICCAI. **Framework:** MONAI Consortium (monai.io) · PyTorch (pytorch.org)

# References


Dataset

[1] Wei, D. et al. (2021). NucMM Dataset: 3D Neuronal Nuclei Instance Segmentation at Sub-Cubic Millimeter Scale. MICCAI 2021. Available at: nucmm-dataset.github.io

U-Net

[2] Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. MICCAI 2015, pp. 234–241.

3D U-Net

[3] Çiçek, Ö., Abdulkadir, A., Lienkamp, S. S., Brox, T., & Ronneberger, O. (2016). 3D U-Net: Learning Dense Volumetric Segmentation from Sparse Annotation. MICCAI 2016, pp. 424–432.

Dice Loss

[4] Milletari, F., Navab, N., & Ahmadi, S. A. (2016). V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation. 3DV 2016, pp. 565–571.

MONAI

[5] MONAI Consortium. (2020). MONAI: Medical Open Network for AI. GitHub. Available at: monai.io · DOI: 10.5281/zenodo.4323925

PyTorch

[6] Paszke, A. et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. NeurIPS 2019, pp. 8026–8037.